

Desenvolvimento para Dispositivos Móveis

Armazenamento remoto

Prof. Chauã Queirolo | <https://github.com/chaaua>

1

O que é o Firestore?

- O Cloud Firestore é um banco de dados NoSQL do Firebase.
- Ele guarda dados na nuvem e permite que o aplicativo acesse esses dados pela internet.
- Diferente de um banco local, os dados não ficam apenas no celular do usuário.

2

Como o Firestore organiza os dados?

O Firestore usa a seguinte estrutura:

coleções -> documentos -> campos

Exemplo:

Coleção: compras

Documento automático: XyZ123
name: "Arroz"

Documento automático: AbC789
name: "Feijão"

Comparação com outros armazenamentos

- SharedPreferences: pequenos dados simples no aparelho
- Hive: objetos locais no aparelho
- Floor: banco local com SQLite
- Firestore: banco online na nuvem

Quando usar Firestore?

- Use Firestore quando os dados precisam ficar fora do celular.
 - lista sincronizada entre dispositivos
 - cadastro de usuários
 - pedidos de uma loja
 - comentários
 - mensagens
 - produtos
 - favoritos de uma conta

Quando não usar Firestore?

- **Não use Firestore para tudo.**
- Para dados simples, como tema escuro ou idioma, `shared_preferences` é mais simples.
- Para dados locais que não precisam de internet, Hive ou Floor podem ser melhores.
- Firestore faz mais sentido quando o dado precisa estar na nuvem.

Dependências necessárias

Para usar Firestore no Flutter, precisamos destes pacotes:

```
firebase_core  
cloud_firestore
```

Instale com:

```
flutter pub add firebase_core  
flutter pub add cloud_firestore  
flutter pub get
```

Para que serve cada pacote?

- **firebase_core** inicializa o Firebase dentro do aplicativo Flutter.
- **cloud_firestore** permite acessar o banco de dados Firestore.
- Sem o **firebase_core**, o app não consegue iniciar os serviços do Firebase.
- Sem o **cloud_firestore**, o app não consegue ler nem salvar dados no Firestore.

Configurando o Firebase

- O projeto Flutter precisa estar conectado a um projeto Firebase.
- O caminho mais comum é usar o FlutterFire CLI:

```
dart pub global activate flutterfire_cli  
firebase login  
flutterfire configure
```

Configurando o Firebase

Esse processo gera ou atualiza o arquivo:

```
lib/firebase_options.dart
```

Ativando o Firestore

- Além do código Flutter, é necessário ativar o Firestore no Firebase Console.
- Caminho básico:

Firebase Console

-  Build
-  Firestore Database
-  Create database

Para aula, pode ser usado o modo de teste.

Em produção, as regras de segurança precisam ser configuradas corretamente.

Inicialização no main.dart

```
void main() async {  
  WidgetsFlutterBinding.ensureInitialized();  
  
  await Firebase.initializeApp(  
    options: DefaultFirebaseOptions.currentPlatform,  
  );  
  
  final rotas = await AppRoutes.getRoutes();  
  
  runApp(AppWidget(rotas));  
}
```

Por que usar WidgetsFlutterBinding?

- Esta linha prepara o Flutter antes de inicializar serviços externos
 - `WidgetsFlutterBinding.ensureInitialized();`
- Ela é importante porque o Firebase precisa ser inicializado antes do aplicativo começar a usar seus recursos.

13

Estrutura da feature

- A feature será organizada em três arquivos:

```
lib/features/shopping_firebase/  
  shopping_model.dart  
  shopping_controller.dart  
  shopping_page.dart
```

14

Responsabilidade dos arquivos

- Cada arquivo tem uma responsabilidade.

`shopping_model.dart` → representa o dado
`shopping_controller.dart` → conversa com o Firestore
`shopping_page.dart` → mostra a tela para o usuário

- Em linguagem simples:

`Model` → o que é o item
`Controller` → o que fazer com o item
`Page` → o que aparece na tela

Criando o model

```
class ShoppingItem {  
  final String id;  
  final String name;  
  
  ShoppingItem({  
    required this.id,  
    required this.name,  
  });  
}
```


Convertendo Firestore para objeto Dart

```
factory ShoppingItem.fromFirestore(  
  Map<String, dynamic> data,  
  String id,  
) {  
  return ShoppingItem(  
    id: id,  
    name: data['name'] ?? '',  
  );  
}
```

17

Convertendo objeto Dart para Firestore

```
Map<String, dynamic> toFirestore() {  
  return {  
    'id': id,  
    'name': name,  
  };  
}
```

18

Convertendo objeto Dart para Firestore

```
Map<String, dynamic> toFirestore() {  
  return {  
    'id': id,  
    'name': name,  
  };  
}
```

19

Criando o controller

- O controller será responsável por:
 - acessar a coleção compras
 - buscar os itens
 - adicionar item
 - remover item
- A tela não deve conversar diretamente com o Firestore.

20

Import do controller

```
import 'package:cloud_firestore/cloud_firestore.dart';  
  
import 'shopping_model.dart';
```

- O primeiro import traz o Firestore.
- O segundo import traz o model ShoppingItem.

Acessando a coleção compras

```
class ShoppingFirestoreController {  
  final CollectionReference<Map<String, dynamic>> _collection =  
    FirebaseFirestore.instance.collection('compras');  
}  
  
// Essa linha indica que a feature vai usar a coleção: compras
```

Buscando itens em tempo real

```
Stream<List<ShoppingItem>> getItems() {  
    return _collection.snapshots().map((snapshot) {  
        return snapshot.docs.map((doc) {  
            return ShoppingItem.fromFirestore(doc.data(), doc.id);  
        }).toList();  
    });  
}
```

23

O que é uma Stream?

- Uma Stream é um fluxo de dados.
- Pense em uma torneira aberta.
- Sempre que o Firestore muda, uma nova lista chega ao aplicativo.

Firestore mudou → Stream envia nova lista → Tela atualiza

24

Salvando um item

```
Future<void> addItem(String name) async {  
    final cleanName = name.trim();  
  
    if (cleanName.isEmpty) return;  
  
    final item = ShoppingItem(  
        id: '',  
        name: cleanName,  
    );  
  
    await _collection.add(item.toFirestore());  
}
```

25

Removendo um item

```
Future<void> deleteItem(String id) async {  
    await _collection.doc(id).delete();  
}
```

26

Criando a page

- A page será responsável por:
 - mostrar o campo de texto
 - mostrar o botão de adicionar
 - mostrar a lista
 - chamar o controller
 - tratar carregamento e erro

27

StatefulWidget

```
class ShoppingFirestorePage extends StatefulWidget {  
  const ShoppingFirestorePage({super.key});  
  
  @override  
  State<ShoppingFirestorePage> createState() =>  
    _ShoppingFirestorePageState();  
}
```

28

Controllers da tela

```
final ShoppingFirestoreController controller =  
    ShoppingFirestoreController();
```

```
final TextEditingController _textController =  
    TextEditingController();
```

- O primeiro controller conversa com o Firestore.
- O segundo controller lê o texto digitado pelo usuário.

Método para adicionar item

```
void _addItem() {  
    controller.addItem(_textController.text);  
    _textController.clear();  
}
```

Método para remover item

```
void _deleteItem(String id) {  
    controller.deleteItem(id);  
}
```

31

Método para remover item

```
void _deleteItem(String id) {  
    controller.deleteItem(id);  
}
```

- A tela não apaga diretamente no Firestore.
- Ela pede para o controller executar a ação.
- Isso deixa o código mais organizado.

32

Liberando memória

```
@override  
void dispose() {  
  _textController.dispose();  
  super.dispose();  
}
```

- Sempre que usamos TextEditingController
 - devemos chamar dispose quando a tela for fechada.

Estrutura visual da tela

```
Scaffold  
  AppBar  
  Column  
    Padding  
      Row  
        TextField  
        IconButton  
      Expanded  
    StreamBuilder
```

Usando StreamBuilder

```
StreamBuilder<List<ShoppingItem>>(  
  stream: controller.getItems(),  
  builder: (context, snapshot) {  
    ...  
  },  
)
```

- O StreamBuilder escuta os dados do Firestore.
- Quando os dados mudam, ele atualiza a interface.

35

Tratando erro

```
if (snapshot.hasError) {  
  return const Center(  
    child: Text('Erro ao carregar os dados.'),  
  );  
}
```

- Esse trecho mostra uma mensagem quando algo dá errado.
- Exemplo: erro de permissão ou problema de conexão.

36

Tratando carregamento

```
if (!snapshot.hasData) {  
  return const Center(  
    child: CircularProgressIndicator(),  
  );  
}
```

- Enquanto os dados ainda não chegaram, mostramos um indicador de carregamento.
- Isso evita que a tela pareça travada.

37

Tratando lista vazia

```
final items = snapshot.data!;  
  
if (items.isEmpty) {  
  return const Center(  
    child: Text('Nenhum item na lista.'),  
  );  
}
```

- Se não houver itens, mostramos uma mensagem clara para o usuário -> Ideal usar uma tela de Empty State

38

Mostrando a lista

```
return ListView.builder(  
  itemCount: items.length,  
  itemBuilder: (context, index) {  
    final item = items[index];  
  
    return ListTile(  
      title: Text(item.name),  
      trailing: IconButton(  
        icon: const Icon(Icons.delete),  
        onPressed: () => _deleteItem(item.id),  
      ),  
    );  
  },  
);
```